

Linux Storage

The Complete A-to-Z Reference

Disks, Partitions, Filesystems, LVM, RAID, Network Storage,
Commands, and Real-World Scenarios

BIG IDEAS. BROKEN INTO BLOCKS.

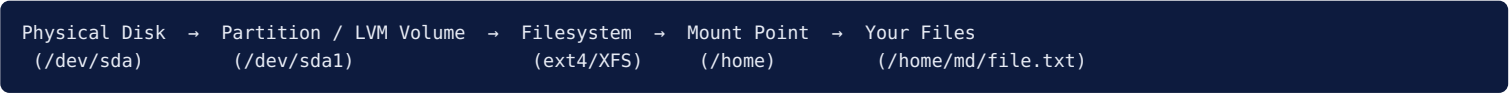
Table of Contents

1. What "Storage" Means in Linux
2. The Storage Stack (How It All Layers)
3. Types of Storage
4. Filesystem Types
5. Partitioning: MBR vs GPT
6. LVM Explained (PV → VG → LV)
7. RAID Explained
8. Network Storage (NFS/SMB/iSCSI)
9. The Complete Command Reference
10. Decoding Command Output, Field by Field
11. When You Should Worry About Storage
12. How to Configure Storage (Best Practices)
13. Real-World Scenarios (Walkthroughs)
14. Acronym Glossary (A-Z)

1. What "Storage" Means in Linux

In Linux, almost everything is represented as a **file** — and storage devices are no exception. A hard disk, SSD, USB drive, or even a RAID array shows up under `/dev/` as a **block device** (e.g. `/dev/sda` , `/dev/nvme0n1`). The kernel talks to these block devices through drivers, and everything you do — partitioning, formatting, mounting — is built on top of that block device abstraction.

The journey of a byte of data looks like this:



Key idea: Linux separates *where data physically lives* (block device) from *how it's organized* (filesystem) from *where you access it* (mount point). Understanding these three separate layers is 80% of understanding Linux storage.

2. The Storage Stack (How It All Layers)

Layer	What It Is	Example
1. Physical/Virtual Disk	The actual hardware or virtual disk presented by hypervisor/cloud	<code>/dev/sda</code> , <code>/dev/nvme0n1</code>
2. Partition Table	Divides the disk into partitions (MBR or GPT)	<code>/dev/sda1</code> , <code>/dev/sda2</code>
3. (Optional) LVM or RAID	Combines/abstracts partitions into flexible logical volumes	<code>/dev/mapper/vg_data-lv_home</code>
4. Filesystem	Organizes raw blocks into files & directories	ext4, XFS, Btrfs
5. Mount Point	Where the filesystem is attached into the directory tree	<code>/</code> , <code>/home</code> , <code>/var</code>

3. Types of Storage

By how it's attached

Type	Acronym	Description	Typical Use
Direct Attached Storage	DAS	Storage physically attached to one server (internal disk, USB, direct SAS)	Single-server workloads, homelabs
Network Attached Storage	NAS	A dedicated file server exposing storage over the network as files (NFS/SMB)	Shared file storage, backups
Storage Area Network	SAN	A dedicated high-speed network presenting raw block devices over Fibre Channel or iSCSI	Enterprise databases, VM datastores
Cloud Block Storage	EBS/PD/etc.	Virtual block device attached to a cloud VM	AWS EC2, GCP Compute Engine disks
Object Storage	S3-compatible	Flat namespace accessed via HTTP API, not mounted as a filesystem	Backups, static assets, data lakes

By physical medium

Type	Notes
HDD (Hard Disk Drive)	Spinning platters, cheap per GB, slower random I/O
SSD (Solid State Drive)	Flash-based, fast, no moving parts
NVMe (Non-Volatile Memory Express)	SSD connected via PCIe instead of SATA — much higher throughput/IOPS

4. Filesystem Types

Filesystem	Best For	Notes
ext4	General purpose, default on many distros	Mature, stable, journaling, supports online resize (grow)
XFS	Large files, RHEL/CentOS default	Excellent parallel I/O performance; can grow online but cannot shrink
Btrfs	Snapshots, checksumming, built-in RAID-like features	Copy-on-write; used by openSUSE, Fedora (optionally)
ZFS	Enterprise NAS, data integrity, snapshots	Combines volume manager + filesystem; not in mainline kernel (licensing)
VFAT/exFAT	USB drives, cross-OS compatibility	No Linux permissions/journaling; used for /boot/efi (VFAT)
swap	Virtual memory extension	Not a real filesystem — a raw partition/file used by the kernel
tmpfs	RAM-backed temp storage	Volatile — cleared on reboot; used for <code>/tmp</code> , <code>/run</code>

5. Partitioning: MBR vs GPT

MBR (Master Boot Record)		GPT (GUID Partition Table)
Max disk size	2 TB	~9.4 ZB (effectively unlimited today)
Max partitions	4 primary (or 3 primary + 1 extended)	128 by default
Boot mode	Legacy BIOS	UEFI (also supports BIOS)
Redundancy	Single copy of partition table	Backup copy stored at end of disk

Rule of thumb for new systems in 2026: always use **GPT** unless you have a specific legacy BIOS requirement.

6. LVM Explained (PV → VG → LV)

LVM (Logical Volume Manager) adds a flexible layer between raw partitions and the filesystem, so you can resize, extend across disks, and snapshot without unmounting.

Term	Acronym	What It Means
Physical Volume	PV	A raw disk or partition initialized for LVM use (e.g. <code>/dev/sdb1</code>)
Volume Group	VG	A pool made of one or more PVs — the "storage pool"
Logical Volume	LV	A "virtual partition" carved out of a VG — this is what you format and mount
Physical Extent	PE	The fixed-size chunk (default 4MB) that a VG is divided into internally
Logical Extent	LE	The LV's view of PEs — LEs map to PEs

Disk(s) → pvcreate → PV → vgcreate → VG → lvcreate → LV → mkfs → mount

Why it matters for you: Since you're already building homelab VMs on ESXi with RHEL, LVM is exactly what lets you grow a VM's `/var` or `/home` later just by adding a virtual disk — no reinstall, no downtime.

7. RAID Explained

Level	Min Disks	Redundancy	Usable Capacity	Notes
RAID 0	2	None	100%	Striping — speed only, one disk failure = total loss
RAID 1	2	Yes	50%	Mirroring — simple, reliable
RAID 5	3	1 disk failure	(n-1)/n	Striping + distributed parity
RAID 6	4	2 disk failures	(n-2)/n	Extra parity, safer than RAID 5
RAID 10	4	Yes	50%	Mirror + stripe — fast and redundant, costly

Linux software RAID is managed with `mdadm` (Multiple Device Admin), producing devices like `/dev/md0` .

8. Network Storage (NFS / SMB / iSCSI)

Protocol	Full Name	Access Style	Typical Use
NFS	Network File System	File-level, native to Unix/Linux	Shared home dirs, app data across Linux servers
SMB/CIFS	Server Message Block	File-level, native to Windows	Mixed Windows/Linux shares
iSCSI	Internet Small Computer Systems Interface	Block-level over TCP/IP	Presents a remote disk as if it were local — used for SAN over Ethernet
FC	Fibre Channel	Block-level over dedicated fabric	High-end enterprise SANs

9. The Complete Command Reference

9.1 Viewing & Identifying Storage

Command	Purpose
<code>lsblk</code>	List block devices as a tree (disks → partitions → LVM/filesystem)
<code>lsblk -f</code>	Same, but shows filesystem type, label, and UUID
<code>blkid</code>	Show UUID and filesystem type for each block device
<code>df -h</code>	Show mounted filesystems and free/used space (human-readable)
<code>df -i</code>	Show inode usage instead of block usage
<code>du -sh /path</code>	Show total size of a directory
<code>fdisk -l</code>	List all disks and partitions (MBR/GPT), needs root
<code>parted -l</code>	Like fdisk but supports GPT more gracefully
<code>mount</code>	Show all currently mounted filesystems (no args)
<code>findmnt</code>	Tree view of mounted filesystems, easier to read than <code>mount</code>
<code>cat /proc/partitions</code>	Raw kernel view of recognized partitions
<code>lsscsi</code>	List SCSI/SATA devices with vendor/model info
<code>smartctl -a /dev/sda</code>	Disk health/SMART data (from smartmontools)

9.2 Partitioning & Formatting

Command	Purpose
<code>fdisk /dev/sdb</code>	Interactive MBR/GPT partition editor
<code>parted /dev/sdb</code>	Interactive partition editor, better for GPT/large disks
<code>mkfs.ext4 /dev/sdb1</code>	Format a partition with ext4
<code>mkfs.xfs /dev/sdb1</code>	Format a partition with XFS
<code>mkswap /dev/sdb2</code>	Initialize a swap partition
<code>swapon /dev/sdb2</code>	Activate swap
<code>fsck /dev/sdb1</code>	Check and repair a filesystem (unmount first!)
<code>e2label /dev/sdb1 mylabel</code>	Set an ext4 filesystem label

9.3 Mounting

Command	Purpose
<code>mount /dev/sdb1 /mnt/data</code>	Mount a device at a directory
<code>umount /mnt/data</code>	Unmount
<code>mount -a</code>	Mount everything listed in <code>/etc/fstab</code>
<code>nano /etc/fstab</code>	Edit persistent mount definitions (survives reboot)
<code>mount -o remount,rw /</code>	Remount a filesystem with new options (e.g. read-write)

9.4 LVM Commands

Command	Purpose
<code>pvccreate /dev/sdb1</code>	Initialize a disk/partition as a Physical Volume
<code>pvddisplay / pvs</code>	Show PV details (verbose / summary table)
<code>vgcreate vg_data /dev/sdb1</code>	Create a Volume Group from one or more PVs
<code>vgextend vg_data /dev/sdc1</code>	Add another disk into an existing VG
<code>vgdisplay / vgs</code>	Show VG details
<code>lvcreate -L 20G -n lv_home vg_data</code>	Create a 20GB Logical Volume inside a VG
<code>lvcreate -l 100%FREE -n lv_home vg_data</code>	Create an LV using all remaining free space
<code>lvdisplay / lvs</code>	Show LV details

<code>lvextend -L +10G /dev/vg_data/lv_home</code>	Grow an LV by 10GB
<code>lvextend -l +100%FREE /dev/vg_data/lv_home</code>	Grow an LV to consume all free VG space
<code>resize2fs /dev/vg_data/lv_home</code>	Grow ext4 filesystem to match the new LV size
<code>xfs_growfs /home</code>	Grow XFS filesystem (works on mount point, not device)
<code>lvreduce -L -5G /dev/vg_data/lv_home</code>	Shrink an LV (ext4 only — never on XFS)
<code>lvremove /dev/vg_data/lv_home</code>	Delete a Logical Volume
<code>lvcreate -s -L 5G -n snap_home /dev/vg_data/lv_home</code>	Create a snapshot of an LV

9.5 RAID (mdadm)

Command	Purpose
<code>mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb1 /dev/sdc1</code>	Create a RAID1 array
<code>mdadm --detail /dev/md0</code>	Show array status and health
<code>cat /proc/mdstat</code>	Quick view of all RAID arrays and sync status
<code>mdadm --add /dev/md0 /dev/sdd1</code>	Add a replacement/spare disk

9.6 Network Storage Mounting

Command	Purpose
<code>mount -t nfs server:/export /mnt/nfs</code>	Mount an NFS share
<code>mount -t cifs //server/share /mnt/smb -o username=md</code>	Mount an SMB/Windows share
<code>showmount -e server</code>	List NFS exports available on a server
<code>iscsiadm -m discovery -t sendtargets -p target_ip</code>	Discover iSCSI targets

9.7 Performance & Monitoring

Command	Purpose
<code>iostat -xz 2</code>	Live disk I/O stats (utilization, wait, throughput)
<code>iotop</code>	Per-process disk I/O, like <code>top</code> for disks
<code>vmstat 2</code>	Includes I/O wait as part of overall system stats
<code>dd if=/dev/zero of=testfile bs=1M count=1024</code>	Simple write-speed benchmark

10. Decoding Command Output, Field by Field

10.1 lsblk columns

```
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda        8:0    0   50G  0 disk
├─sda1     8:1    0    1G  0 part /boot
├─sda2     8:2    0   49G  0 part
└─vg_data-lv_root 253:0  0   49G  0 lvm  /
```

Field	Meaning
NAME	Device name (indentation shows parent-child: disk → partition → LVM)
MAJ:MIN	Kernel major:minor device numbers
RM	Removable (1 = yes, e.g. USB)
SIZE	Total capacity
RO	Read-only flag
TYPE	<code>disk</code> = whole disk, <code>part</code> = partition, <code>lvm</code> = LVM logical volume, <code>rom</code> = optical drive
MOUNTPOINTS	Where it's currently mounted, blank if not mounted

10.2 df -h columns

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/vg-lv_root   49G   12G   35G  26% /
```

Field	Meaning
Filesystem	The block device or mapper path backing this mount
Size	Total filesystem capacity
Used / Avail	Space used vs. still free
Use%	Percentage used — the number to watch ; alert above ~80%
Mounted on	Mount point in the directory tree

10.3 fdisk -l fields

Field	Meaning
Disk /dev/sda: 50 GiB	Whole disk device and total size
Units	Sector size used for calculations (usually 512 bytes)
Disklabel type	<code>dos</code> = MBR, <code>gpt</code> = GPT
Device	Partition device name (e.g. <code>/dev/sda1</code>)
Boot	Asterisk (*) marks the bootable partition
Start / End	Sector range of the partition
Sectors / Size	Partition length
Id / Type	Partition type code (e.g. 83 = Linux, 8e = LVM, 82 = swap)

10.4 pvs / vgs / lvs summary columns

Tool	Key Columns	Meaning
<code>pvs</code>	PV, VG, Fmt, Attr, PSize, PFree	Device, which VG it belongs to, LVM format, attributes, total/free size on this PV
<code>vgs</code>	VG, #PV, #LV, #SN, Attr, VSize, VFree	VG name, count of PVs/LVs/snapshots, attributes, total/free capacity in the pool
<code>lvs</code>	LV, VG, Attr, LSize, Pool, Origin, Data%	LV name, parent VG, attributes (writable/active/snapshot state), size, and % used (for thin pools/snapshots)

11. When You Should Worry About Storage

Red flags that need action, not just observation:

- **Disk usage above 80-85%** on any filesystem via `df -h` — plan expansion before it hits 95%+
- **Inode exhaustion** (`df -i` shows 100% even though space looks free) — common when millions of tiny files exist
- **High I/O wait** (`%iowait` in `vmstat / top` consistently above ~20-30%) — disk is a bottleneck
- **SMART errors or reallocated sectors** (`smartctl -a`) — disk is physically failing
- **Degraded RAID array** (`cat /proc/mdstat` shows `[U_]` instead of `[UU]`) — one disk is down, no more redundancy
- **LVM snapshot nearing full** — a full snapshot becomes invalid and unusable
- **Read-only filesystem remount** in `dmesg` — the kernel detected corruption and protected itself
- **Swap usage climbing steadily** — may indicate memory pressure, not a storage problem per se, but shows up as disk I/O

12. How to Configure Storage (Best Practices)

1. **Separate partitions/LVs for volatile data:** keep `/var`, `/var/log`, `/tmp`, and `/home` on their own LVs so runaway logs can't fill up `/` and crash the whole system.
2. **Always use LVM on servers/VMs** unless you have a specific reason not to — it turns "I need more space" from a reinstall into a five-minute online operation.
3. **Use UUIDs in `/etc/fstab`, not `/dev/sdX` names** — device letters can shift across reboots, especially with multiple disks; UUIDs don't.
4. **Leave free space in the Volume Group** when initially sizing LVs, rather than allocating 100% immediately — gives you room to grow without adding a new disk.
5. **Pick the filesystem for the job:** XFS for large files/databases and easy online growth; ext4 for general-purpose flexibility (including shrink support).
6. **Set sensible mount options:** `noatime` to reduce write overhead on busy filesystems; `nodev,nosuid,noexec` on things like `/tmp` for security.
7. **Monitor proactively** — set alert thresholds (e.g. via Prometheus `node_exporter`, Zabbix, or a simple cron + `df` script) rather than discovering "disk full" from a crashed service.
8. **For redundancy-critical data, use RAID1/RAID10 or cloud-managed redundant volumes** — RAID0 and single disks are both single points of failure.
9. **Test backups, not just take them** — an untested backup is a hope, not a plan.

13. Real-World Scenarios (Walkthroughs)

Scenario 1 — Extending a Full "/" Partition on a Live RHEL VM (no downtime)

Situation: Your homelab RHEL VM's root LV is at 92% and a Foreman/Katello sync just failed due to lack of space. The VM is on ESXi and you can add virtual disk capacity without rebooting.

1. Extend the virtual disk in ESXi (or add a new virtual disk)
2. Rescan the SCSI bus so Linux sees the new size:
`# echo 1 > /sys/class/scsi_device/*/device/rescan`
3. If it's a new disk, partition and initialize it:
`fdisk /dev/sdb # create sdb1`
`pvccreate /dev/sdb1`
`vgextend vg_data /dev/sdb1`
4. Grow the Logical Volume:
`lvextend -l +100%FREE /dev/vg_data/lv_root`
5. Grow the filesystem online:
`xfs_growfs / # for XFS`
`resize2fs /dev/vg_data/lv_root # for ext4`
6. Confirm:
`df -h /`

Scenario 2 — Building Redundant Storage for a Homelab NAS/Foreman Content Store

Situation: You want your Katello content storage to survive a single disk failure without manual restore.

1. Identify two equal-size disks: `/dev/sdb, /dev/sdc`
2. Create a RAID1 array:
`mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc`
3. Put LVM on top of the RAID device (best practice – RAID for redundancy, LVM for flexibility):
`pvccreate /dev/md0`
`vgcreate vg_content /dev/md0`
`lvcreate -l 100%FREE -n lv_content vg_content`
4. Format and mount:
`mkfs.xfs /dev/vg_content/lv_content`
`mount /dev/vg_content/lv_content /var/lib/foreman`
5. Verify redundancy status any time:
`cat /proc/mdstat`

Scenario 3 — Sharing an Ansible Control Node's Directory via NFS to RHEL Client VMs

Situation: You want Ansible playbooks/artifacts on one host, mounted read-only across your homelab VMs.

- ```
-- On the NFS server --
1. Install: dnf install nfs-utils
2. Export the directory in /etc/exports:
 /srv/ansible 192.168.1.0/24(ro,sync,no_subtree_check)
3. Apply and start:
 exportfs -ra
 systemctl enable --now nfs-server

-- On each RHEL client VM --
4. Discover exports:
 showmount -e 192.168.1.10
5. Mount:
 mount -t nfs 192.168.1.10:/srv/ansible /mnt/ansible
6. Make it persistent in /etc/fstab:
 192.168.1.10:/srv/ansible /mnt/ansible nfs ro,defaults 0 0
```

## Scenario 4 — Diagnosing "Disk Full" When df Says There's Space

**Situation:** An application refuses to write files with "No space left on device," but `df -h` shows 40% free.

1. Check inodes, not just blocks:  
`df -i`

- # If IUse% is 100%, you've run out of inodes (too many small files)
- 2. Check for deleted-but-open files still holding space:  
lsof +L1
- 3. Check for a separate, smaller partition being the actual bottleneck  
(e.g. /var full while / has room):  
df -h /var /var/log /tmp
- 4. Find what's eating space:  
du -sh /var/\* | sort -rh | head -10
- 5. Fix: rotate/clear logs, remove old kernels, or extend the LV  
(see Scenario 1) if it's a genuine capacity issue rather than  
an inode/log problem.

# 14. Acronym Glossary (A-Z)

| Acronym  | Full Form                                          | Quick Meaning                                      |
|----------|----------------------------------------------------|----------------------------------------------------|
| DAS      | Direct Attached Storage                            | Disk physically attached to one server             |
| FC       | Fibre Channel                                      | Dedicated high-speed block storage network         |
| GPT      | GUID Partition Table                               | Modern partitioning scheme, replaces MBR           |
| HBA      | Host Bus Adapter                                   | Card connecting a server to SAN/storage fabric     |
| HDD      | Hard Disk Drive                                    | Spinning magnetic storage                          |
| iSCSI    | Internet SCSI                                      | Block storage over standard IP networks            |
| IOPS     | Input/Output Operations Per Second                 | Measure of storage performance                     |
| JBOD     | Just a Bunch Of Disks                              | Disks presented individually, no RAID              |
| LE       | Logical Extent                                     | LVM's logical-side chunk, maps to a PE             |
| LV       | Logical Volume                                     | The "virtual partition" you format/mount, from LVM |
| LVM      | Logical Volume Manager                             | Flexible layer for resizing/pooling storage        |
| MBR      | Master Boot Record                                 | Legacy partitioning scheme, 2TB/4-partition limit  |
| NAS      | Network Attached Storage                           | File-level storage shared over a network           |
| NFS      | Network File System                                | Unix/Linux-native network file sharing protocol    |
| NVMe     | Non-Volatile Memory Express                        | High-speed SSD interface over PCIe                 |
| PE       | Physical Extent                                    | Fixed-size chunk a VG is divided into              |
| PV       | Physical Volume                                    | A disk/partition initialized for LVM               |
| RAID     | Redundant Array of Independent Disks               | Combining disks for speed and/or redundancy        |
| SAN      | Storage Area Network                               | Dedicated network presenting block-level storage   |
| SMB/CIFS | Server Message Block / Common Internet File System | Windows-native network file sharing protocol       |
| SSD      | Solid State Drive                                  | Flash-based storage, no moving parts               |
| UUID     | Universally Unique Identifier                      | Stable ID for a filesystem, used in fstab          |
| VG       | Volume Group                                       | Pool of storage made from one or more PVs          |
| XFS      | (originally) eXtents File System                   | High-performance filesystem, default on RHEL       |